MyCodeSchool

while și funcții

Fișa 2 · funcții cu parametri, while, cifrele unui număr, return

Ai voie să folosești: tot ce ai din fișa 1 (variabile, `int` / `float` / `str` / `bool`, `print` și f-string, `input` cu `int()`, `+ - * / // % **`, comparații, `and or not`, `if / else`); `def nume(n):` cu parametri; `while`, inclusiv un `while` în alt `while`; `+= -= *= //=`; `return` (cu o valoare, cu `True` / `False` sau cu `None`); o funcție care cheamă altă funcție.
Reține: o problemă = o funcție, cu numele și parametrii din enunț, în fișierul din enunț. Problemele 1–25 stau în `exercitii_while.py` și **afișează**; problemele 26–54 stau în `functii.py` și **întorc** valoarea cu `return`, iar `main()` o afișează. Rulezi `main.py` cu codul din „În main()"; ce afișează trebuie să fie identic cu ieșirea așteptată, inclusiv mesajele de la `input`. În blocuri, **verde** = ce tastezi tu.
Încă NU folosim: `for`, `range`, liste, `break`, transformarea numărului în text (`str(n)`, `len`): cifrele le scoți doar cu `% 10` și `// 10`.

Înainte de problema 1 — proiectul tău

Proiectul are trei fișiere. În `exercitii_while.py` și `functii.py` funcțiile au deja numele și parametrii, iar corpul e `pass`. Un parametru e o variabilă pe care o primește funcția când e chemată: la `afiseaza_de_la_1_la(5)`, în corpul funcției `n` este `5`.

Programul pornește din `main.py`: aduce funcțiile cu `from exercitii_while import ...` și `from functii import ...` și le cheamă în `main()`. La fiecare problemă adaugi numele funcției în `import` și pui în `main()` codul din „În main()".

exercitii_while.py

```
def afiseaza_de_la_1_la(n):
    pass
```

main.py

```
from exercitii_while import afiseaza_de_la_1_la
from functii import ultima_cifra


def main():
    afiseaza_de_la_1_la(5)
    print(ultima_cifra(4738))


if __name__ == "__main__":
    main()
```

Cum rulezi în PyCharm: rulezi mereu `main.py`: ▶ verde de lângă `if __name__ == "__main__":`, sau click dreapta în `main.py` → **Run 'main'** (Windows: `Ctrl+Shift+F10` · Mac: `Ctrl+Shift+R`). Rezultatul apare jos, în fereastra **Run**; acolo tastezi și valorile cerute de `input`, apoi `Enter`. Cât timp o funcție din `functii.py` are doar `pass`, `print(ultima_cifra(4738))` afișează `None`: funcția nu întoarce încă nimic.

Partea 1 — la clasă

Pornești de la starter: fiecare funcție are deja numele și parametrul, iar corpul e `pass`. Urmezi pașii în ordine.

1. De la 1 la n

while

În `exercitii_while.py`: `def afiseaza_de_la_1_la(n):`

Afișează numerele de la 1 la `n`, câte unul pe rând.

1. Pornește un contor: `i = 1`.
2. Scrie `while i <= n:`. Cât timp condiția e `True`, Python repetă rândurile indentate sub ea.
3. În buclă: `print(i)`, apoi `i += 1`. Fără `i += 1` condiția rămâne mereu adevărată și bucla nu se mai oprește.

În main()

```
afiseaza_de_la_1_la(5)
```

Ieșire așteptată

```
1
2
3
4
5
```

2. Numărătoarea inversă

while

În `exercitii_while.py`: `def afiseaza_numaratoare_inversa(n):`

Afișează numerele de la `n` în jos până la 1, apoi `Start!`.

1. Contorul pornește de la `n`, iar condiția e `i >= 1`.
2. În buclă afișezi `i` și îl scazi: `i -= 1`.
3. `print("Start!")` stă **după** buclă, la nivelul lui `while`, ca să se afișeze o singură dată.

În main()

```
afiseaza_numaratoare_inversa(3)
```

Ieșire așteptată

```
3
2
1
Start!
```

În main()

```
afiseaza_numaratoare_inversa(0)
```

Ieșire așteptată

```
Start!
```

3. Numerele pare

while

În `exercitii_while.py`: `def afiseaza_pare_pana_la(n):`

Afișează numerele pare de la 2 până la `n`.

1. Contorul pornește de la 2.
2. Crește-l cu 2 la fiecare pas: `i += 2`. Așa sari peste numerele impare fără niciun `if`.

În main()

```
afiseaza_pare_pana_la(10)
```

Ieșire așteptată

```
2
4
6
8
10
```

În main()

```
afiseaza_pare_pana_la(7)
```

Ieșire așteptată

```
2
4
6
```

4. Tabla înmulțirii

while + f-string

În `exercitii_while.py`: `def afiseaza_tabla_inmultirii(n):`

Afișează tabla înmulțirii cu `n`, de la 1 la 10.

1. Contorul merge de la 1 la 10, nu până la `n`.
2. Fiecare rând e un f-string: `f"{n}x{i}={n * i}"`.

În main()

```
afiseaza_tabla_inmultirii(7)
```

Ieșire așteptată

```
7x1=7
7x2=14
7x3=21
7x4=28
7x5=35
7x6=42
7x7=49
7x8=56
7x9=63
7x10=70
```

5. Din 5 în 5

while

În `exercitii_while.py`: `def afiseaza_din_5_in_5(n):`

Afișează multiplii lui 5 până la `n`.

1. Același tipar ca la problema 3: alege de unde pornește contorul și cu cât crește.

În main()

```
afiseaza_din_5_in_5(23)
```

Ieșire așteptată

```
5
10
15
20
```

6. Suma de la 1 la n

acumulare

În `exercitii_while.py`: `def afiseaza_suma_pana_la(n):`

Calculează suma 1 + 2 + … + n și afișează `Suma este ...`.

1. Înainte de buclă: `suma = 0`.
2. În buclă adaugi contorul la sumă: `suma += i`.
3. După buclă: `print(f"Suma este {suma}")`.

În main()

```
afiseaza_suma_pana_la(5)
```

Ieșire așteptată

```
Suma este 15
```

În main()

```
afiseaza_suma_pana_la(100)
```

Ieșire așteptată

```
Suma este 5050
```

7. Câte numere pare

contor + if

În `exercitii_while.py`: `def afiseaza_cate_pare_pana_la(n):`

Numără câte numere pare sunt de la 1 la `n` și afișează `Sunt ... numere pare`.

1. Înainte de buclă: `contor = 0`; contorul buclei `i` merge de la 1 la `n`, din 1 în 1.
2. În buclă: `if i % 2 == 0:` → `contor += 1`. `i += 1` rămâne în afara lui `if`.

În main()

```
afiseaza_cate_pare_pana_la(10)
```

Ieșire așteptată

```
Sunt 5 numere pare
```

În main()

```
afiseaza_cate_pare_pana_la(7)
```

Ieșire așteptată

```
Sunt 3 numere pare
```

8. Ultima cifră și ce rămâne

% și //

În `exercitii_while.py`: `def afiseaza_ultima_cifra_si_rest(n):`

Fără buclă: afișează ultima cifră a lui `n` și numărul care rămâne după ce o tai.

1. `n % 10` e ultima cifră, `n // 10` e numărul fără ea.
2. Afișezi `Ultima cifra: ...`, apoi `Ce ramane: ...`.

În main()

```
afiseaza_ultima_cifra_si_rest(4738)
```

Ieșire așteptată

```
Ultima cifra: 8
Ce ramane: 473
```

9. Cifrele, de la coadă

while pe cifre

În `exercitii_while.py`: `def afiseaza_cifrele(n):`

Afișează cifrele lui `n`, de la ultima la prima, câte una pe rând.

1. `while n > 0:` — afișezi `n % 10`, apoi tai cifra: `n //= 10`.
2. Pentru `n = 0` bucla nu pornește deloc; un `if n == 0:` înainte de ea afișează `0`.

În main()

```
afiseaza_cifrele(4738)
```

Ieșire așteptată

```
8
3
7
4
```

În main()

```
afiseaza_cifrele(0)
```

Ieșire așteptată

```
0
```

10. Câte cifre are

while pe cifre

În `exercitii_while.py`: `def afiseaza_numarul_de_cifre(n):`

Afișează câte cifre are `n`: `Are ... cifre`.

1. Orice număr are cel puțin o cifră: `contor = 1`.
2. Cât timp `n >= 10` (mai are cel puțin două cifre): tai una și crești contorul.

În main()

```
afiseaza_numarul_de_cifre(4738)
```

Ieșire așteptată

```
Are 4 cifre
```

În main()

```
afiseaza_numarul_de_cifre(0)
```

Ieșire așteptată

```
Are 1 cifre
```

Partea 2 — de exersat

Aceleași tipare, combinate. De aici primești doar cerința și exemplele.

11. Suma cifrelor

exersat

În `exercitii_while.py`: `def afiseaza_suma_cifrelor(n):`

Afișează suma cifrelor lui `n`.

În main()

```
afiseaza_suma_cifrelor(4738)
```

Ieșire așteptată

```
Suma cifrelor este 22
```

12. Cifra maximă

exersat

În `exercitii_while.py`: `def afiseaza_cifra_maxima(n):`

Afișează cea mai mare cifră a lui `n`.

În main()

```
afiseaza_cifra_maxima(4738)
```

Ieșire așteptată

```
Cifra maxima este 8
```

În main()

```
afiseaza_cifra_maxima(500)
```

Ieșire așteptată

```
Cifra maxima este 5
```

13. Cifra minimă

exersat

În `exercitii_while.py`: `def afiseaza_cifra_minima(n):`

Afișează cea mai mică cifră a lui `n`.

În main()

```
afiseaza_cifra_minima(4738)
```

Ieșire așteptată

```
Cifra minima este 3
```

În main()

```
afiseaza_cifra_minima(905)
```

Ieșire așteptată

```
Cifra minima este 0
```

14. Câte cifre pare

exersat

În `exercitii_while.py`: `def afiseaza_cate_cifre_pare(n):`

Afișează câte cifre pare are `n`. Cifra 0 e pară, deci `0` are o cifră pară.

În main()

```
afiseaza_cate_cifre_pare(4738)
```

Ieșire așteptată

```
Are 2 cifre pare
```

În main()

```
afiseaza_cate_cifre_pare(0)
```

Ieșire așteptată

```
Are 1 cifre pare
```

În main()

```
afiseaza_cate_cifre_pare(1357)
```

Ieșire așteptată

```
Are 0 cifre pare
```

15. Oglinditul

exersat

În `exercitii_while.py`: `def afiseaza_oglindit(n):`

Afișează numărul scris cu cifrele în ordine inversă. Zerourile de la final dispar în față.

În main()

```
afiseaza_oglindit(4738)
```

Ieșire așteptată

```
Oglindit: 8374
```

În main()

```
afiseaza_oglindit(1200)
```

Ieșire așteptată

```
Oglindit: 21
```

16. Palindrom

exersat

În `exercitii_while.py`: `def afiseaza_palindrom(n):`

Un număr e palindrom dacă e egal cu oglinditul lui. Afișează `Este palindrom` sau `Nu este palindrom`.

În main()

```
afiseaza_palindrom(12321)
```

Ieșire așteptată

```
Este palindrom
```

În main()

```
afiseaza_palindrom(4738)
```

Ieșire așteptată

```
Nu este palindrom
```

17. Pașii până la o cifră

exersat

În `exercitii_while.py`: `def afiseaza_pasii_radacinii(n):`

Cât timp `n` are mai mult de o cifră, îl înlocuiești cu suma cifrelor lui și afișezi pasul. La final afișezi cifra la care ai ajuns.

În main()

```
afiseaza_pasii_radacinii(199)
```

Ieșire așteptată

```
199 -> 19
19 -> 10
10 -> 1
Rezultat final: 1
```

În main()

```
afiseaza_pasii_radacinii(7)
```

Ieșire așteptată

```
Rezultat final: 7
```

18. Doar cifrele pare

exersat

În `exercitii_while.py`: `def afiseaza_cifrele_pare(n):`

Formează numărul din cifrele pare ale lui `n`, în aceeași ordine, și afișează-l. Dacă nu are nicio cifră pară, afișează `Nu are nicio cifra para`.

În main()

```
afiseaza_cifrele_pare(4738)
```

Ieșire așteptată

```
Cifrele pare formeaza: 48
```

În main()

```
afiseaza_cifrele_pare(2046)
```

Ieșire așteptată

```
Cifrele pare formeaza: 2046
```

În main()

```
afiseaza_cifrele_pare(1357)
```

Ieșire așteptată

```
Nu are nicio cifra para
```

Citire până la 0. Problemele 19–25 n-au parametri: citesc numere întregi de la tastatură până când se tastează `0`, iar `0` nu intră în calcul. Citești o dată **înainte** de `while` și încă o dată la **sfârșitul** blocului; condiția buclei e `!= 0`.

19. Media notelor

exersat

În `exercitii_while.py`: `def citeste_note_si_media():`

Citește note cu mesajul `"Nota: "` până la 0. Afișează câte note s-au introdus și media lor, sau `Nu s-a introdus nicio nota`.

În main()

```
citeste_note_si_media()
```

Ieșire așteptată

```
Nota: 9
Nota: 10
Nota: 8
Nota: 0
Ai introdus 3 note
Media este 9.0
```

```
Nota: 0
Nu s-a introdus nicio nota
```

20. Suma numerelor citite

exersat

În `exercitii_while.py`: `def citeste_suma_numerelor():`

Citește numere cu mesajul `"Introduceti un numar: "` până la 0. Afișează câte numere s-au introdus și suma lor, sau `Nu s-a introdus niciun numar`.

În main()

```
citeste_suma_numerelor()
```

Ieșire așteptată

```
Introduceti un numar: 5
Introduceti un numar: -2
Introduceti un numar: 7
Introduceti un numar: 0
Au fost introduse 3 numere
Suma lor este 10
```

```
Introduceti un numar: 0
Nu s-a introdus niciun numar
```

21. Câte pozitive

exersat

În `exercitii_while.py`: `def citeste_cate_pozitive():`

Același mesaj de citire, până la 0. Afișează câte numere pozitive s-au introdus.

În main()

```
citeste_cate_pozitive()
```

Ieșire așteptată

```
Introduceti un numar: 5
Introduceti un numar: -2
Introduceti un numar: 7
Introduceti un numar: -1
Introduceti un numar: 0
Au fost introduse 2 numere pozitive
```

22. Câte negative

exersat

În `exercitii_while.py`: `def citeste_cate_negative():`

Același mesaj de citire, până la 0. Afișează câte numere negative s-au introdus.

În main()

```
citeste_cate_negative()
```

Ieșire așteptată

```
Introduceti un numar: 3
Introduceti un numar: -4
Introduceti un numar: -6
Introduceti un numar: -1
Introduceti un numar: 0
Au fost introduse 3 numere negative
```

23. Suma pozitivelor

exersat

În `exercitii_while.py`: `def citeste_suma_pozitivelor():`

Același mesaj de citire, până la 0. Afișează suma numerelor pozitive.

În main()

```
citeste_suma_pozitivelor()
```

Ieșire așteptată

```
Introduceti un numar: 5
Introduceti un numar: -2
Introduceti un numar: 7
Introduceti un numar: 0
Suma numerelor pozitive este 12
```

24. Maximul

exersat

În `exercitii_while.py`: `def citeste_maximul():`

Același mesaj de citire, până la 0. Afișează cel mai mare număr introdus, sau `Nu s-a introdus niciun numar`. Merge și când toate numerele sunt negative.

În main()

```
citeste_maximul()
```

Ieșire așteptată

```
Introduceti un numar: -3
Introduceti un numar: -8
Introduceti un numar: -1
Introduceti un numar: 0
Numarul maxim este -1
```

```
Introduceti un numar: 0
Nu s-a introdus niciun numar
```

25. Minimul

exersat

În `exercitii_while.py`: `def citeste_minimul():`

Același mesaj de citire, până la 0. Afișează cel mai mic număr introdus, sau `Nu s-a introdus niciun numar`.

În main()

```
citeste_minimul()
```

Ieșire așteptată

```
Introduceti un numar: 4
Introduceti un numar: 9
Introduceti un numar: 2
Introduceti un numar: 6
Introduceti un numar: 0
Numarul minim este 2
```

Înainte de problema 26 — funcții care întorc

Funcțiile din `functii.py` nu afișează nimic: calculează o valoare și o **întorc** cu `return`. Apelul `dublu(21)` devine valoarea întoarsă, pe care `main()` o afișează sau o folosește mai departe. `return` oprește funcția pe loc. O funcție din `functii.py` poate chema altă funcție din același fișier, scrisă deasupra ei.

exemplu

```
def dublu(n):
    return n * 2
```

în main()

```
print(dublu(21))
print(dublu(dublu(5)) + 1)
```

ieșire

```
42
21
```

26. Ultima cifră

return

În `functii.py`: `def ultima_cifra(n):`

Întoarce ultima cifră a lui `n`.

În main()

```
print(ultima_cifra(4738))
print(ultima_cifra(5))
```

Ieșire așteptată

```
8
5
```

27. Prima cifră

return

În `functii.py`: `def prima_cifra(n):`

Întoarce prima cifră a lui n.

În main()

```
print(prima_cifra(4738))
print(prima_cifra(7))
```

Ieșire așteptată

```
4
7
```

28. Numărul de cifre

return

În `functii.py`: `def numar_cifre(n):`

Întoarce câte cifre are n.

În main()

```
print(numar_cifre(4738))
print(numar_cifre(0))
```

Ieșire așteptată

```
4
1
```

29. Cifre pare

return

În `functii.py`: `def numar_cifre_pare(n):`

Întoarce câte cifre pare are n (0 e cifră pară).

În main()

```
print(numar_cifre_pare(4738))
print(numar_cifre_pare(0))
```

Ieșire așteptată

```
2
1
```

30. Cifre impare

return

În `functii.py`: `def numar_cifre_impare(n):`

Întoarce câte cifre impare are n. Folosește cele două funcții de mai sus.

În main()

```
print(numar_cifre_impare(4738))
print(numar_cifre_impare(1357))
```

Ieșire așteptată

```
2
4
```

31. Suma cifrelor

return

În `functii.py`: `def suma_cifrelor(n):`

Întoarce suma cifrelor lui n.

În main()

```
print(suma_cifrelor(4738))
```

Ieșire așteptată

```
22
```

32. Produsul cifrelor

return

În `functii.py`: `def produsul_cifrelor(n):`

Întoarce produsul cifrelor lui n.

În main()

```
print(produsul_cifrelor(4738))
print(produsul_cifrelor(105))
print(produsul_cifrelor(0))
```

Ieșire așteptată

```
672
0
0
```

33. Cifra maximă

return

În `functii.py`: `def cifra_maxima(n):`

Întoarce cea mai mare cifră a lui n.

În main()

```
print(cifra_maxima(4738))
```

Ieșire așteptată

```
8
```

34. Cifra minimă

return

În `functii.py`: `def cifra_minima(n):`

Întoarce cea mai mică cifră a lui `n`.

În main()

```
print(cifra_minima(4738))
```

Ieșire așteptată

```
3
```

35. Are cifra?

return bool

În `functii.py`: `def are_cifra(n, cifra):`

Întoarce `True` dacă `cifra` apare în `n`, altfel `False`.

În main()

```
print(are_cifra(4738, 7))
print(are_cifra(4738, 5))
```

Ieșire așteptată

```
True
False
```

36. De câte ori apare

return

În `functii.py`: `def de_cate_ori_apare(n, cifra):`

Întoarce de câte ori apare `cifra` în `n`.

În main()

```
print(de_cate_ori_apare(1213, 1))
print(de_cate_ori_apare(1213, 5))
print(de_cate_ori_apare(0, 0))
```

Ieșire așteptată

```
2
0
1
```

37. Suma cifrelor pare

return

În `functii.py`: `def suma_cifrelor_pare(n):`

Întoarce suma cifrelor pare ale lui `n`.

În main()

```
print(suma_cifrelor_pare(4738))
```

Ieșire așteptată

```
12
```

38. Produsul cifrelor pare

return None

În `functii.py`: `def produsul_cifrelor_pare(n):`

Întoarce produsul cifrelor pare ale lui `n`. Dacă `n` nu are nicio cifră pară, întoarce `None`.

În main()

```
print(produsul_cifrelor_pare(4738))
print(produsul_cifrelor_pare(1357))
print(produsul_cifrelor_pare(0))
```

Ieșire așteptată

```
32
None
0
```

39. Cifra de pe o poziție

return

În `functii.py`: `def cifra_de_pe_pozitia(n, pozitie):`

Pozițiile se numără de la dreapta: poziția 1 e ultima cifră. Întoarce cifra de pe `pozitie`, sau −1 dacă poziția nu există.

În main()

```
print(cifra_de_pe_pozitia(4738, 1))
print(cifra_de_pe_pozitia(4738, 4))
print(cifra_de_pe_pozitia(4738, 5))
```

Ieșire așteptată

```
8
4
-1
```

40. Oglinditul

return

În `functii.py`: `def oglindit(n):`

Întoarce oglinditul lui `n`.

În main()

```
print(oglindit(4738))
print(oglindit(1200))
```

Ieșire așteptată

```
8374
21
```

41. Este palindrom? `return bool`

În `functii.py`: `def este_palindrom(n):`

Întoarce `True` dacă `n` e palindrom. Folosește `oglindit`.

În main()

```
print(este_palindrom(12321))
print(este_palindrom(4738))
```

Ieșire așteptată

```
True
False
```

Partea 3 — provocări

Doar cerința. Poți chema orice funcție scrisă mai sus în `functii.py`.

42. Cifre distincte `provocare`

În `functii.py`: `def are_cifre_distincte(n):`

Întoarce `True` dacă nicio cifră nu apare de două ori în `n`.

În main()

```
print(are_cifre_distincte(4738))
print(are_cifre_distincte(1213))
```

Ieșire așteptată

```
True
False
```

43. Cifre crescătoare `provocare`

În `functii.py`: `def are_cifre_crescatoare(n):`

Întoarce `True` dacă cifrele lui `n`, citite de la stânga la dreapta, sunt strict crescătoare.

În main()

```
print(are_cifre_crescatoare(1379))
print(are_cifre_crescatoare(1337))
print(are_cifre_crescatoare(7))
```

Ieșire așteptată

```
True
False
True
```

44. Lipește două numere `provocare`

În `functii.py`: `def lipeste(a, b):`

Întoarce numărul format din cifrele lui `a` urmate de cifrele lui `b`.

În main()

```
print(lipeste(12, 345))
print(lipeste(7, 0))
```

Ieșire așteptată

```
12345
70
```

45. Suma pe pozițiile impare `provocare`

În `functii.py`: `def suma_pozitii_impare(n):`

Pozițiile se numără de la dreapta, de la 1. Întoarce suma cifrelor de pe pozițiile 1, 3, 5, …

În main()

```
print(suma_pozitii_impare(4738))
```

Ieșire așteptată

```
15
```

46. Suma pe pozițiile pare `provocare`

În `functii.py`: `def suma_pozitii_pare(n):`

La fel, pentru pozițiile 2, 4, 6, …

În main()

```
print(suma_pozitii_pare(4738))
```

Ieșire așteptată

```
7
```

47. Prima și ultima `provocare`

În `functii.py`: `def prima_si_ultima(n):`

Întoarce numărul de două cifre format din prima și ultima cifră a lui `n`.

În main()

```
print(prima_si_ultima(4738))
print(prima_si_ultima(7))
```

Ieșire așteptată

```
48
77
```

48. Câte cifre diferite

provocare

În `functii.py`: `def cate_cifre_diferite(n):`

Întoarce câte cifre diferite apar în `n`.

În main()

```
print(cate_cifre_diferite(1213))
print(cate_cifre_diferite(4738))
```

Ieșire așteptată

```
3
4
```

49. Toate cifrele identice

provocare

În `functii.py`: `def toate_cifrele_identice(n):`

Întoarce `True` dacă toate cifrele lui `n` sunt la fel.

În main()

```
print(toate_cifrele_identice(777))
print(toate_cifrele_identice(4738))
```

Ieșire așteptată

```
True
False
```

50. Rădăcina digitală

provocare

În `functii.py`: `def radacina_digitala(n):`

Cât timp `n` are mai mult de o cifră, îl înlocuiești cu suma cifrelor lui. Întoarce cifra la care ajungi.

În main()

```
print(radacina_digitala(199))
print(radacina_digitala(4738))
```

Ieșire așteptată

```
1
4
```

51. Fără cifrele pare

provocare

În `functii.py`: `def fara_cifrele_pare(n):`

Întoarce numărul format din cifrele impare ale lui `n`, în aceeași ordine (0 dacă nu rămâne nicio cifră).

În main()

```
print(fara_cifrele_pare(4738))
print(fara_cifrele_pare(2468))
```

Ieșire așteptată

```
73
0
```

52. Cifre comune

provocare

În `functii.py`: `def cifre_comune(a, b):`

Întoarce câte cifre diferite apar și în `a`, și în `b`.

În main()

```
print(cifre_comune(4738, 1873))
print(cifre_comune(12, 34))
```

Ieșire așteptată

```
3
0
```

53. Cel mai mare număr

provocare

În `functii.py`: `def cel_mai_mare_numar(n):`

Întoarce cel mai mare număr care se poate forma cu toate cifrele lui `n`.

În main()

```
print(cel_mai_mare_numar(4738))
print(cel_mai_mare_numar(1020))
```

Ieșire așteptată

```
8743
2100
```

54. Numărătoarea

provocare

În `functii.py`: `def numaratoarea():`

Citește numere cu mesajul `"n = "` până la 0. Afișează câte au toate cifrele identice și, dacă există, pe cel mai mare dintre ele.

În main()

```
numaratoarea()
```

Ieșire așteptată

```
n = 777
n = 12
n = 5
n = 3333
n = 0
Cu toate cifrele identice: 3
Cel mai mare dintre ele: 3333
```

```
n = 12
n = 0
Cu toate cifrele identice: 0
```